INDEX

2407010999 - CSE
3rd Yr.

NAME: Deekshitha. STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: _____

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
1.	20/7	STUDY OF NETWORK COMMANDS -	1.	28/8/24
2.	2/8	types of network cables.	7	8
3.	4/8	packet tracer tool install	9	9
4.	4/8	setting up LAN	11	8
5.	13/9	wireshark.	14	10
6.	13/9	Hamming code	16	9
7.	14/9	sliding window	19	9
8.	16/9	NMAP live host discovery ARP. ICMP & TCP/UDP ping scans.	22	9
9.	19/9	subnetting in CISCO	25	9
10-a	22/9	interconnecting with router in CISCO	27	9
10-b	22/9	interconnecte using wireless router	29	9
11-a.	26/9	static routing config in cisco	30	9
11-b.	26/9	simulate using cisco packet tracing.	31	9

S. No.	Date	Title	Page No.	Teacher's Sign / Remarks
12.	30/9	End to end communication at transport layer ~~layer~~	9	30/9
13.	30/9	implement ping program	6	30/9
14.	30/9	code using RAW Sockets to implement packet sniffing.	9	30/9

Ex. no. 1

STUDY OF NETWORK COMMANDS

Windows commands:

1. arp -a : ARP is a short term of address resolution protocol.

O/P

Interface : 10.16.56.50 ... 0x9.

Internet address	Physical address
10.16.56.155	ec-12-17-17-12-52
10.16.56.255	ff-ff-ff-ff-ff-ff

2. hostname - this is the simplest of all TCP/IP command. It will display the name of your computer.

O/P: LAPTOP-T003BKTN

3. IPconfig /all : this command displays configuration. information about your TCP/IP connation.

O/P

window IP configuration.

Host name LAPTOP-T003BKTN
Node type ..... mixed.

4. nbstat -a: this command helps solves problems with netBIOS name resolution.

O/P: Ethernet 2:
Node IP address : [192.168.56.1]-
Scope ID : []
Host not found.

5. netstat: (Network statistics) Netstat display a variety of statistics about a computer active TCP/IP connection.

O/P: active connection.

proto	local address	foreign address	state.
TCP	10.16.56.50.4.9.444	4.213.25.242: https	Established

6. nslookup UP: It is used to display the DNS detail such as IP address of a particular computer

O/P:
server: unknown
address: 10: 16: 56: 155

7. pathping: unique to windows and its basically a combination of the ping and tracert commands.

O/P: and: pathping g. google. com.
route to google.com [142.250.66.14]
over a max. of 30 hops.

8. Ping: way to connectivity between two nodes.

O/P: ~~Ping~~ google.com
pinging.

9. Route: its used to show or manipulate the IP routing table. It is primarily used to setup static routes to specific tag.

LINUX COMMANDS

1. IP commands.

1. Cmd: IP address show - displays an IP address of network interfaces.

O/P:
eth0: inet 192.168.1.10/24.

2. IP address add 192.168.1.254/24 dw eth0 - assign an IP address to an interface.

O/P: IP address added successfully.

3. IP link set eth0 up - enables a network interface.

O/P: eth0 state UP.

4. IP link set eth0 down - disables the network interface.

O/P: eth0 state down.

5. IP link set eth0 promisc on - Enable promiscuous mode.

O/P - promiscuous mode enabled.

6 IP route added default via 192.168.1.254 dw eth0 - adds the default gateway.

7. IP route get 10.10.1.4 - shows the route to a destination IP.

O/P: 10.10.1.4 via 192.168.1.254 dwi eth0

* If config - displays the network interface.

O/P: etho link en cop: Ethernet inc
192.168.1.10

* mtr

→ mtr google.com - shows route statics to a host

O/P:

HOST: localhost 100% Avg Bu.

→ mtr -g google.com - shows numeric ip address only.

O/P: 142.250.183.286.

→ mtr -b google.com - shows both username and IP address

O/P: goog google.com (142.250.183.206)

→ mtr -c 10 google.com - sends only 10 pronic.

O/P: sent 10

⟹ tcpdump.

→ dnf install -g tcpdump - install tcpdump.

→ tcpdump -D - the available capture interface

O/P: 1.etho 2.10 modules

→ tcpdump -i etho -c -10 capture packets from a host:

O/P 10 packets captured.

→ tcpdump -i etho src host 8.8.8.8. - Capture packet from a host.

O/P: 8.8.8.8 > 192.168.1.10

→ tcpdump -i etho dst host 8.8.8.8. Captures to a host.

O/P: 192.168.1.10 > 8.8.8.8.

6. tcpdump -i eth0. port 53 ... DNS traffic

o/p: DNS only captured.

→ tcpdump -i eth0 -c 10 host www.google.com and port 443 - captures HTTPS packets.

o/p: TCP 443 captured.

→ tcpdump -i eth0 port not 53 and not 25 - capture all traffic except DNS and SMTP

o/p: TCP/IP packets displayed.

⟹ PING.

Ping google.com - check connectivity to a host.

Ping -c 10 google.com - sends only 10 ping requests. o/p: 10 packets transmitted.
- 10 received.

⟹ nmcli:

→ nmcli connection show - used to all network connections.

o/p: wired connection

→ nmcli connection modify "wired connection" modify an existing connection.

→ nmcli connection show - display connection details.

o/p:
IPV4 . method auto.

→ nmcli connection modify "wired connection 1". ipv4.method auto - configures DHCP.

O/P - ipv4 method set to auto

→ nmcli connection modify wired connection 1. ipv4.method manual ipv4.address 192.0.2.1/24. ipv4.gateway 192.0.2.254 ipv4.dns 192.0.2.200 - configures a static IPv4 address

op ip v4 settings upgraded.

Results:

The above both windows commands & linux commands are executed and outputs are compiled.

28/8/24

Ex. no. 2

TYPES OF NETWORK CABLES

AIM

To study of different types of network cables.

a. understand different types of network cable different types of cables used in networking area.

1. unshielded twisted pair (UTP) cable
2. shielded twisted pair (STP) cable
3. coaxial cable.
4. fibre optic cable.

cable type	category	Max data trans	advantages / disadvantages	Application / use.	Image
UTP	3	10 bps	advantage cheaper in cost easy to install. astmy have small diameter dis. more prone to (emi) electromagnetic interference.	10 base T ethernet fast ethernet Gigabit ethernet.	
	5	upto 100bps			
	5e	1gbps			
STP	6, 6a	10gbps	adv. shielded faster than UTP dis. expensive greater installation effort.	gigabit ethernet 10G ethernet (55m) widely used gigabit ethernet 10 g ethernet (100m)	
SSTP	7	10Gbps			

coaxial cable	RG-6 RG 59 RG 11.	10-100 mbps	high bandwidth. immune to interference. low loss / bandwidth. adv limited distance cost size is bulky.	speed of signal is 500m. TV network high speed internet connection.	
fibre optics cable.	single mode multi mode	100 mbps	adv high speed high bandwidth immunity. long distance disadv expensive requires skill	maximum distance of fibre optics cable is around 100 m.	

b) make your own ethernet cross-over cable / straight cable.

Tools and parts needed.

→ ethernet cabling CAT 5e is electrified for gigabits support ~~but~~ CAT 5 cabling works as well.

→ A crimping tool. this is all in one networking tool shaped to push down the pining the ping & strip & cut the shielding of the cables.

Result

Both type of network cables & making cross wired & straight cable using clamping

Ex. no. 3 :

11/8/24

PACKET tracer tool installation and user interface overview.

AIM.

To study the packet tracer tool installation and user interface overview.

Analyse the behaviours of network device using CISCO PACKET TRACER simulator

1. From the network component box.
click & drag & drop the below components.
   a. 4 generic pcs, and one HUB
   b. 4 generic pcs & one switch.

2. click on connections.
   a. click on copper straight - through cable
   b. select one of the pc & connect it to HUB using the cable.
   c. similarly connect 4 pcs to the switch using copper straight through cable.

3. click on the pcs connected. hub & switch go to the desktop hub click on IP config & enter an IP address that starts with 10.1.1.1.

4. Drag & drop the PDU icon in one pc (same) & drop it on another pc (destination) connected to hubs & switch.

5. Observe the flow of PDU from source PC to destination PC by selecting the realtime mode of simulation.

6. Observe how HUB & switches are forwarding PDU

RESULT:

Thus, the packet tracer tool using cisco successfully executed.

11/8/22

Ex. no. 4.

Setting up a local Area Network.

AIM.

To design setup and configure a LAN connecting 4 component through a network switch, assign static IP address to all devices, verify connectivity using ping command and demonstrate resource or data sharing across network.

Equipment required.

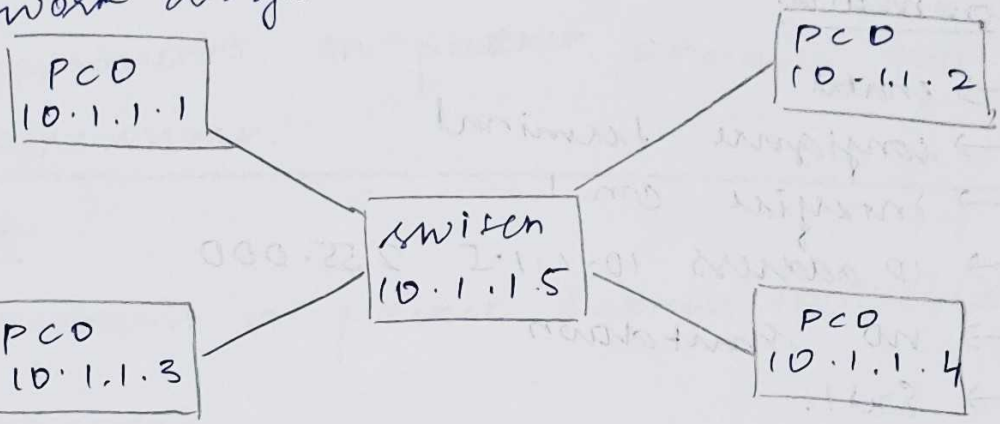
Sl.no.	Device.	Qty.	type.
1.	(PC) personal computer.	4.	ethernet enabled.
2.	network switch	1	8/16/24 port, unmanaged / managed
3.	ethernet cables.	4.	straight thru RJ-45 connectors.

theory:

→ LAN is a network that connects computers & devices within limited area like home, office.

→ star topology → all end devices are individually connected to central switch.

Network diagram.



IP	Address.	configuration.	Table.	Port interface
PC0	10.1.1.1	255.0.0.0	switch	1 fa 0/1
PC1	10.1.1.2	255.0.0.0	switch	2 fa 0/2
PC2	10.1.1.3	255.0.0.0	switch	3 fa 0/3
PC3	10.1.1.4	255.0.0.0	switch	4 fa 0/4
switch.	10.1.1.5	255.0.0.0	LAN	5 interface

Implementation of cisco packet tracer.

Step.1

Draw the devices open cisco packet tracer select 4PC's onto workshop-select & drag and one switch.

Step.2

Cable the device.

click connection → copper straight wire & connect all 4PCs to switch viva.

Step.3.

assign IP address to Pcs

Step.4

click the switch → CLI tab enters following command.

Command

→ enable
→ configure terminal
→ interface con 1
→ IP address 10.1.1.5 255.000
→ no shutdown
→ Exit.

Step 5

verify connectivity using pins
click pc → desktop → command prompt use ping command to test connectivity.

eg - ping 10.1.1.2

Step 6

Demonstrate data sharing
Add a switch device & connect it to switch, assign it IP open server switch

Step 7

observe simulation mode

Result.

Thus the using appear local area network (LAN) was implemented and executed successfully

13/8/25

Ex.no:5 13/2/26

Experiments on packet capture tool wireshark.

AIM.

Experiment on packet capture tool: wireshark

WIRESHARK:

wireshark is a network analysis tool formerly known as Ethereal, capture packet in real time and display them in human readable format.
it includes filters colour coding and other feature that let dig deep

CAPTURING PACKET

→ after downloading & installing wireshark launch it & download.
→ select local area connecting in wireshark
→ go to capture option.
→ select stop capture automatically
→ save the packets.

FILTERING PACKETS

→ Goto capture option.
→ select stop capture automatically after 100 packets.
→ then click start capture
→ search ARP packet in search box
→ save the packet.

Result

Capturing & filtering packet using wireshark is done successfully.

13/8

Ex. no. 6. 13/9/26

Hamming code.

AIM

wise program to implement error diction and correction using hamming code. concept make test sum to input data stream & verify correcting.

CODE:

```
dy calc_redundant_bits(m)
    for r in range (m+1):
        if (2**r) >= (m+r+1):
            return r;

dy main():
    data = input("Enter data min : "). m
    m = (in Cdata)
    r = calc_redundant_bin(m)
    n = m+r
    print(f" number of r mudd : {r} ")
    print (f" total frame dungtha : {n}")
    coa = ['0'] + (n+1).
    j = 0
    for i in range (1, n+1):
        if (id (i-1) != 0
            coa [i] = data [-(j+1)]
            j += 1
    for i in rang (r):
        baing_bcs = 2 ++ i
        rai = 0
        for pos in range (i, n+1)
        if Lpcs & parity p(s) != 0
```

14/9/24

Ex.no.7

SLIDING WINDOW.

AIM

To write the sliding window protocol code using python.

CODE:

```
Sender.py.
import time.
import os
ws. input (input ("window size"))
msg = input ("Enter message")
frame = [(i+1, ch), for i ch in
enter (msg')
i = 0
while i < len (frames);
    window = frame [i = i + os]
    with open ("sender_buffer", "w") as:
    for no, data in window)
    time. sleep(2)
    while not os. path. exists ("Recv Buffer")
    time sleep (1)
    with open ("receiver_Buffer") as f
    ack, no f read (). strip () split ();
    no = int (no)
print ("received, ackno).
if ack = "Ack" and no == window
E i] [0) ;
    i + = ws
    dof if ack == "Nack":
```

```
i = n -
open("receiver Buffer", "w").close()
print("In Transmission completed")
```

receiver.py

```
import time,os
expected = 1
while True
    if not os.path.exists("sender.txt")
    time.sleep(1)
    continue
    with open("sender Buffer") as f:
    frames.[ (int(x.split(";")[0])),
x.split(",")[1].strip() for x in f
    n.strip()
    if not frames:
        time.sleep(1)
        continue
    print("\n Received ; frames)
    for no.data in frames:
    if no! .exented.
        f.write(f"Ack {ex expected}")
    expected += 1
else
with open("Reciever_Buffer", "w") as
f.write(f"Ack = {frame[-1][0]}")
time - sleep().
```

output

sender:

window size 3 Enter message

sending [('1','v')(2,'i')(3,'3')

recieved ACK 5

sending [(4,'n')(5,'2')]

recieved ACK 4

transmission completed.

reciever

recieved ('v','i'),(2,'i')(3,'3').

recieved [(4,'n'),(5,'4')])

Result:

thus, the code is implemented for sliding window

Ex.no.8 16/9/26

N.map live Host Discovery. ARP. ICMP & TCP/UDP ping scans.

AIM

To discover live hosts on a network using Nmap host-discovery scans-ARP scan, ICMP scan and TCP/UDP ping scan, and to understand the protocol operate which layer in TCP/IP model.

ARP Host Discovery.

sudo nmap -PR -sn 10.2006.0/24
starting nmap 7.80 (https://mag.org)
Nmap scan. port for 10.2006.0
host is up.
Nmap done 256 ip address (1 host up)
scanned in 1051

ICMP host discovery

Icmp echo (-PE)
$ sudo nmap -PE -sn 10.200.6.0/24
NMAP. scan. report for 10.200.6.1
host is up (0.0.69. latency)
Nmap scan report for 0.6.250

icmp timestamp (-PP)

$nmap -PP -sn 10.200.6 -0/24.
nmap scan report for 10.200.6.50
host. is up (0.0.69 latency)
Nmap scan report for 10.200.6.250
host in up (0.0.69, latency)

ICMP address mask (-PM)

$ nmap -PM -sn 10.200.60/24.

Nmap done 256 IP address (0 hosts up) scanned in 52.14 seconds.

TCP & UDP ping discovery

TCP SYN ping (-PS)

$ nmap -PS -sn 10.200.6.0/24

host is up (0.067s latency).

Nmap done: 256 IP address.

TCP ACK ping (-PA)

$ sudo nmap -PA -sn 10.200.6.0/24

Nmap scan report for 10.200.6.50

host is up (0.070s latency)

Nmap scan report for 10.200.625

Result:

Different routing algorithms were successfully identified using different map.

16/9/24

Ex. no. 9

Implementation of subnetting in CISCO PACKET TRACER simulator.

AIM.

To implement & subnetting in cisco pa
trace simulator.

Procedure

1. open cisco and create a new network
2. add the required router, switches &
PCs to the workspace
3. connect the device using app. appropriate ca
4. use the network 192.168.10/24. and
divide 2 networks.
5. configure the router interface with the
received IP address & subnet mask
255.255.255.224.
6. use enable and configure terminal rout
7. enter the interface configuration
8. configure the required switch po
in access mode
9. assign each pc its pc address
subnet mask and default gateways
9. repeat the configuration for all
required subnets.
10. use the ping command to pc &
routers.
11. if ping is successful the network
configured correctly.

Result:

Thus the implementation of subnetting in cisco packet tracer simulator is successful.

19/12/24

Ex. 10.a

22/9/26

Interworking with router with in cisco packet tracer simulator.

AIM

To design and configure a simple internetwork using a router in cisco packet tracer and fast connectivity b/w two pcs.

Procedure:

Step 1 — configure router.

* drag the router and open CLI
* click to start configuring ports.
* type enable to activate the privilege mode

Step 2

* assign IP address to every pc on the network.
* select the PC, go to desktop & select IP config, and assign and IP address.
* assign the default gateways of PC0 as 192.168.10.1.
*

Step 3

* connect fast Ethernet 0 port of PC0 with fast ethernet 0 port of router using a copper straight through cable.

Ex. 11.a

26/9/26

Simulate static routing configuration using cisco packet tracer.

AIM

To simulate a static routing configuration using cisco packet tracer

Procedure.

* open cisco packet tracer and create the given network topology
* connect 3 routers and PC's suitably.
* configure IP address for router and PC.
* enable all router interface using no shut down
* configure static routes on router 0, router 1, router 2 using IP route.
* if main and backup route using different administrative distance value
* test connectivity.
* use trace to verify the routing path & discounts.
* thus a static routing is successfully configured and verified.

Ex. no. 11.b

26/9/25

Simulate RIP using cisco packet tracing.

Aim:

To simulate RIP using cisco packet tra

Procedure:

1. Make the topology with 3 routers & 2 pc
2. connect the devices using suitable cab
3. assign the given IP address to both
4. configure fast ethernet & serial interfaces of all routers.
5. Enable interfaces using router RIP.
6. add directly connected network using the network command.
7. test connectivity using ping 20.
8. use traces 20.0.0.2. to verify the routed routers.
9. Disconnect the device routers.

Result:

Thus static routing configuration RIP simulated using cisco packet tracer.

Ex. no. 10.b. 22/9/24

Design and configure an internetwork using wireless router, DHCP server and internet cloud.

Aim.

To design and configure an internetwork using a wireless router. DHCP server and internet cloud in cisco packet tracer.

PROCEDURE:

1. Open cisco packet tracer and create the required network topology.
2. Connect PC, laptop, wireless router
3. Configure the wireless router with home network and enable DHCP
4. Connect the laptop to the wireless network.
5. Configure the server with IP 205.67.220.2 and enable DHCP and DNS.
6. Verify connectivity using TP config and ping cisco.com.

Result:

The internetwork was successfully designed and configured using a router. DHCP server and internet cloud and connectivity was successful and pong.

Ex. no. 12 27/9/24

End to End communication at Transport layer.

a. Implement echo client server using TCP/UDP sockets.

TCP:

```
def start_server.
server = socket = socket(socket.AF_INET
socket.SOCK_STREAM)
server.
```

```
import socket
HOST = "127.0.0.1"
PORT = 5000
client = socket.socket(socket.AF_INET,
                       socket.SOCK_STREAM)
client.connect((HOST, PORT))
print("connected to TCP Echo server.")
print("Type 'exit' to quit.")
while True:
    message = input("Enter message: ")
    client.sendall(message.encode())
    if message.strip().lower() == 'exit':
        break
    data = client.recv(1024)
    print("Echo from server:", data.
                                decode()).
client.close()
```

UDP server.

```
import socket
HOST = "127.0.0.1"
PORT = 5000.
server = socket.socket(socket.AF_INET,
        socket.SOCK_DGRAM)
server.bind ((HOST, PORT))
print ("UDP Echo Server is running...")
while True:
data, addr = server.recvfrom(1024)
message = data.decode()
if message.strip().lower() == "exit":
    print("Client disconnected:", addr)
    continue
print("Client:", message)
server.sendto (data, addr).
```

UDP client.

```
import socket.
HOST = "127.0.0.1"
client = socket.socket(socket.AF_INET, SOCK
print ("connected to UDP echo server.")
print ("Type 'exit' to quit.")
while True:
    message = input("enter message
    client.sendto (message.encode(),
        (HOST, PORT)).
```

Thanks

if message.strip().lower() == "exit":
break.

data, addr = client.recvfrom(1024)
print("Echo from server:", data.decode())
client.close().

OUTPUT

TCP

[server] listening on 127.0.0.1:5000
[client] connected to server.
hellow.
[server] received: hello.
[server response] server received: hello

VDP

enter msg (or 'quit' to stop): [server]
Running on 172.0.0.1.1200hi
[server] recied from ("127.0.0.1", 49670): hi
Echo from server: hi

Result:

The end to end communication and transportation was successful.

29/9/24

30/9/26

Ex.no 13

Implement your own ping program.

AIM:

To implement ping program.

code

```
Import os
import platform.
import subprocess
def ping(host = "google.com", count=4):
    param = "-n" if platform.system().
        lower() == "windows" else "-c".
    command = ["ping", param, str(count),
                                    ]
try:
    output = subprocess.check_output
        (command, universal_newlines = True)
    print(output)
except Exception as e:
    print(f"ping failed: {e}")
if __name__ == "__main__":
    ping("8.8.8.8")
```

Output:

pinging 8.8.8.8 with 32 bytes of data

reply from 8.8.8.8: bytes = 32 time = 6ms TTL = 114

reply from 8.8.8.8: bytes = 32 time 9ms TTL = 119

reply from 8.8.8.8: bytes = 32 time = 5ms TTL = 119

reply from 8.8.8.8: bytes = 32 time = 9ms TTL = 119

ping statistics for 8.8.8.8

packets: sent = 4 received = 4, Lost = 0 (0% loss),

approx. round trip times in ms:

minimum = 5 ms, maximum = 9ms.

Average = 7ms.

Result:

Therefore implementation of my own ping program is successful.

30/9/20

30/9/24

Ex. 14.

Write a code using RAW sockets to implement packet sniffing.

code:

```
from scapy.all import sniff.
from scapy.layers.inet import IP, TCP, UDP
def packet_callback(packet):
    if IP in packet:
        ip_layer = packet[IP]
        protocol = ip_layer.proto
        src_ip = ip_layer.src
        dst_ip = ip_layer.dst

        # Determine the protocol
        protocol_name = "
        if protocol == 1:
            protocol_name = "ICMP"
        elif protocol == 6:
            protocol_name = "TCP"
        elif protocol == 17:
            protocol_name = "UDP"
        else:
            protocol_name = "unknown protocol"

        # print packet details.
        print(f"protocol: {protocol_name}").
```

```
print(f"source IP: {src_ip}")
print(f"destination IP: {dst_ip}")
print("-" * 50)
```

Output:

```
protocol : TCP
source IP : 57.155.141.117.
destination IP 192.168.1.6.
- - - - - - - - - - -
protocol : TCP
source IP : 192.168.1.6
destination IP: 57.155.141.117.
```

Result:

Thus for a code using RAW SOCKETS to implement packet sniffing is successfully executed.

30/9/25